北京理工大学新生助手项目中期汇报：开发进度与开发框架详细总结

1. 项目目标与当前范围

1.1 项目目标

本项目面向北京理工大学新生，建设一个可通过微信公众号访问的智能服务系统，核心目标包括：

- 提供地点查询能力（地点检索、地图定位、静态地图预览）。
- 提供新生常见问题智能问答能力（FAQ 检索 + 可选大模型生成）。
- 形成可部署、可测试、可演示、可运维的工程化交付版本。

1.2 本阶段范围（中期）

当前中期版本聚焦功能 1 与功能 2：

- 功能 1（地点查询）：已完成。
- 功能 2（AI 问答）：已完成可用版本（含 FAQ 与 DeepSeek 接入、网页演示模式）。
- 功能 3（人工服务工单系统）：尚未进入开发。

2. 总体架构与模块拆分

2.1 总体架构

系统采用“前端页面 + Python 后端 + Nginx 反向代理 + 微信公众号入口”的结构：

- 统一入口页面：`/`。
- 地点查询页面：`/place`。
- 智能问答页面：`/chat`。
- 问答 API：`/api/chat`。
- 检索调试 API：`/api/kb_query`。
- 演示状态 API：`/api/demo_status`。
- 微信回调接口：`/wechat`（GET 校验签名，POST 处理消息）。

2.2 代码目录与职责

- `feature1-place/`
  - 纯前端地点查询页面，调用腾讯地图 JS API 与 WebService API。
- `feature2-ai/frontend/`
  - `index.html`：功能导航主页。
  - `chat.html`：智能问答交互页（手动提问 + 来源展示）。
- `feature2-ai/backend/`
  - `app.py`：Flask 主服务、路由、问答流程、微信接口。
  - `kb.py`：向量检索构建与查询（Sentence-Transformers + FAISS）。
  - `requirements.txt`：后端依赖。
- `feature2-ai/data/`
  - `faq.md` 及扩展知识文档：当前共 13 份 `.md` 文档。
- `deploy/`
  - Nginx、systemd、Windows 服务部署参考文档。

3. 技术栈选型与选型原因

3.1 后端技术栈

- Python + Flask
  - 选择原因：
    - 上手快、学习曲线平滑，适合课程项目节奏。
    - 对微信回调、HTTP API、快速迭代极友好。
    - 与数据处理、RAG 生态兼容性高。
- requests + python-dotenv
  - 选择原因：
    - 轻量稳定，便于调用外部模型 API。
    - 使用 `.env` 管理密钥与配置，便于开发/部署分离。

3.2 RAG 检索技术栈

- sentence-transformers（嵌入模型）
- faiss-cpu（向量索引）
- numpy（向量计算）

选择原因：

- 在课程项目中可快速构建“可解释的检索增强问答”原型。
- FAISS 在中小规模知识库场景下性能与实现复杂度平衡较好。
- 与纯 FAQ 匹配机制形成“向量检索 + 规则兜底”的双层策略，增强稳定性。

3.3 大模型接入技术栈

- OpenAI 兼容协议调用（HTTP）
- 当前优先配置：DeepSeek API（ `https://api.deepseek.com` + `deepseek-chat` ）

选择原因：

- OpenAI 兼容格式降低了供应商切换成本。
- DeepSeek 在中文问答任务上表现良好、接入门槛低。
- 通过统一变量与优先级策略（ `DEEPSEEK_*` 优先， `OPENAI_*` 回退）支持后续扩展。

3.4 前端技术栈

- 原生 HTML/CSS/JavaScript（无框架）

选择原因：

- 交付快、依赖少、部署简单。
- 适合公众号 H5 场景，维护成本低。
- 对演示与答辩友好，可直观展示功能链路。

3.5 部署技术栈

- Nginx（TLS、反向代理）
- Gunicorn（WSGI 进程管理）
- systemd（服务托管与开机自启）
- Ubuntu 24.04 LTS（服务器环境）

选择原因：

- 方案成熟、文档完善、便于排障。
- 满足微信公众号对公网 HTTPS 的要求。
- 能形成教学项目可复现的工程化部署标准。

4. 当前开发进度（已完成 / 未完成）

4.1 已完成内容

A. 功能 1：地点查询（已完成）

- 已实现地点关键词查询。
- 已实现地图中心定位与标记展示。
- 已实现地点名称、地址、坐标显示。
- 已实现静态地图图像预览。
- 已实现空输入、无结果、Key 异常的提示处理。

B. 功能 2：智能问答（已完成可用版）

- 已完成 FAQ 文档读取与基础问答兜底。
- 已完成向量检索（FAISS）查询接口。
- 已完成 LLM 增强回答链路（检索结果注入提示词）。
- 已完成 DeepSeek API 接入并实测返回结果。
- 已完成统一前端聊天页与 `POST /api/chat` 交互。
- 已完成 `GET /api/kb_query` 供检索/调试验证。
- 已新增 `GET /api/demo_status` 供现场演示前自检。

C. 网页演示能力（本次新增）

- 前端已整合为“地图 + 问答”统一入口与双栏展示。
- 聊天页保留手动提问模式，回答显示来源与置信信息。
- 后端提供 `/api/demo_status` 用于演示前环境自检（索引状态/模型状态）。

D. 微信接入链路（基础可用）

- 已实现微信 GET 签名校验。
- 已实现微信 POST 文本消息接收与回复。
- 已实现非文本消息快速返回 success。

E. 部署与测试文档（持续更新中）

- 已形成 Ubuntu + Nginx + Gunicorn + systemd 的部署说明。
- 已形成功能 1+2 的测试步骤与清单。

4.2 未完成内容

A. 功能 3：人工服务工单系统（未开发）

- 新生提问工单流转。
- 学长学姐后台接单与回复。
- 状态管理（待处理/处理中/已完成）。
- 用户通知与闭环反馈。

B. 工程化增强（部分未完成）

- 自动化测试（单元测试/接口测试）尚未建立完整体系。
- 统一日志分级与告警机制尚未完善。
- 生产级并发与压测数据尚未形成。
- 权限控制与后台管理台尚未实现。

C. 数据与模型优化（部分未完成）

- 知识库已扩展到 13 份文档，但仍需持续扩充与清洗。
- RAG 分块策略仍较基础，待优化（语义分块、重排等）。
- 大模型回答质量评估指标体系尚未系统化。

5. 关键实现思路与设计决策

5.1 问答策略：检索优先 + 生成增强 + 兜底

当前问答流程采用三层设计：

1. 优先执行知识库检索（`KB_FORCE_LEXICAL=0` 时优先向量检索）。
2. 将检索结果注入提示词，调用 LLM 进行整理回答。
3. 若 LLM 不可用或失败，则退回 FAQ/兜底回答。

这样可以保证：

- 在 API 异常或配额不足时系统仍有可用回复。
- 回答可附带来源片段，增强可解释性。
- 兼顾准确性与服务连续性。

实测补充：

- 在 5000 端口环境中， `/api/demo_status` 可返回 `kb_backend=faiss` 、 `kb_doc_count=13` 。
- `/api/rebuild_index` 首次重建约 34 秒返回 200，属于可接受的模型加载与索引构建耗时。

5.2 模型供应商切换策略

通过环境变量优先级实现：

- 优先读取 `DEEPSEEK_*` 。
- 若未提供 DS key，再回退 `OPENAI_*` 。

该策略降低了未来切换模型平台的改造成本，符合“接口兼容优先”的工程原则。

5.3 演示友好性设计

考虑中期汇报场景，新增演示支持：

- 演示前可调用 `/api/demo_status` 快速确认配置状态。
- 演示中以“手动提问 + 来源展示”突出可解释问答能力。
- 演示后可执行 `/api/rebuild_index` 展示知识库热更新能力。

6. 当前风险与应对策略

6.1 外部 API 风险

风险：

- DeepSeek Key 失效、余额不足、网络波动。

应对：

- 保留 FAQ 兜底路径，避免完全不可用。
- 暴露演示状态接口，提前发现环境问题。
- 可配置超时参数，必要时提高超时时间。

6.2 知识库覆盖不足风险

风险：

- FAQ 规模不足导致“无法确定”比例较高。

应对：

- 建立 FAQ 持续补录机制。
- 对高频问题优先补充权威答案。

6.3 部署一致性风险

风险：

- 本地与服务器环境差异导致“本地可用、线上异常”。

应对：

- 使用统一部署文档与 systemd/Nginx 固化流程。
- 发布前执行标准化测试步骤与清单。

7. 下一阶段计划（面向结题）

7.1 功能层

- 开始功能 3（人工服务工单系统）需求落地与数据库设计。
- 完成新生端/学长端页面与工单流转 API。
- 增加问题状态追踪与消息通知机制。

7.2 工程层

- 增加基础自动化测试（关键 API 冒烟 + 回归）。
- 完善日志结构化与错误码体系。
- 补充运行监控与异常告警。

7.3 数据与模型层

- 扩展 FAQ 知识源，形成可版本化知识库。
- 优化 RAG 检索质量（分块、召回、重排）。
- 建立回答质量评估集并定期复测。

8. 中期结论

截至当前阶段，项目已完成“可演示、可部署、可在公众号中接入”的核心形态：

- 功能 1 与功能 2 已具备实际可用性。
- 后端具备检索增强与模型增强的双路径能力。
- 前端具备汇报场景下的一键演示和环境自检能力。
- 部署与测试文档已形成并与当前进度同步。

项目下一阶段重点应从“功能可用”转向“系统完整与工程质量提升”，即补全功能 3，并建设测试、监控、数据迭代等长期能力。